VŠB TECHNICKÁ UNIVERZITA OSTRAVA | FAKULTA ELEKTROTECHNIKY A INFORMATIKY

# Komponenta výukového serveru TI - P-úplné problémy

Component of Teaching Server for Theoretical Computer Science - P-complete problems

Tomáš Kirnig

Bakalářská práce

Vedoucí práce: Ing. Martin Kot, Ph.D.

Ostrava, 2025

Č.j.VSB/25/116933
vsboee25037FA8

# Zadání bakalářské práce

Student: **Tomáš Kirnig**

Studijní program: B0613A140014 Informatika

Téma: Komponenta výukového serveru TI - P-úplné problémy
Component of Teaching Server for Theoretical Computer Science - P-complete problems

Jazyk vypracování: čeština

Zásady pro vypracování:

V rámci diplomových a bakalářských prací vzniká výukový server pro předměty teoretické informatiky. Jedná se o sadu dynamických webových stránek umožňujících studentům pochopení různých typů úloh a problémů. Na rozdíl od běžných výukových textů s pevně daným počtem ukázkových příkladů umí tyto stránky generovat libovolně mnoho ukázek na základě vstupů od uživatele. Cílem této konkrétní bakalářské práce je vytvořit komponentu pro pomoc s výukou tzv. P-úplných problémů.

Vytvořte dynamické webové stránky umožňující uživateli následující:
1. Simulovat výpočet řešení problému Monotone Circuit Value Problem (MCVP) a alespoň 2 dalších P-úplných problémů.
2. Vstupy těchto algoritmů bude moci uživatel zadávat třemi způsoby:
a) Vhodným, uživatelsky přívětivým, způsobem ručně.
b) Nechat si vstup vygenerovat zcela náhodně podle nastavených parametrů.
c) Vybrat z předpřipravené sady vhodně zvolených vstupů.
3. Bude možné si zobrazit převod instance problému MCVP na ty dva zvolené P-úplné problémy. Přitom:
a) Instanci MCVP pro převod bude možné zadat kterýmkoliv z výše uvedených způsobů.
b) Převod si bude moci uživatel krokovat se zobrazením slovního vysvětlení jednotlivých kroků.
c) Na převodem vytvořenou instanci bude opět možné použít výše požadovanou simulaci výpočtu řešení.

Seznam doporučené odborné literatury:

[1] Miyano, S., Shiraishi, S., Shoudai, T.: "A List of P-Complete Problems", Kyushu University, RIFIS-TR-CS-17, December 1990, dostupné z URL: https://catalog.lib.kyushu-u.ac.jp/opac_download_md/3123/rifis-tr-17.pdf
[2] Sawa, Z.: "Teoretická informatika", podklady pro přednášky, VŠB - Technická univerzita Ostrava, dostupné z URL: https://www.cs.vsb.cz/sawa/ti/slides/ti-slides-03.pdf
[3] Papadimitriou, C.: Computational Complexity, Addison Wesley, 1993
[4] Arora, S., Barak, B.: Computational Complexity: A Modern Approach, Cambridge University Press, 2009

Formální náležitosti a rozsah bakalářské práce stanoví pokyny pro vypracování zveřejněné na webových stránkách fakulty.

Vedoucí bakalářské práce: **Ing. Martin Kot, Ph.D.**

Datum zadání: 01.09.2024

Datum odevzdání: 30.04.2026

Garant studijního programu: Ing. Tomáš Fabián, Ph.D.

V IS EDISON zadáno: 18.08.2025 09:42:23

## Abstrakt

Tato bakalářská práce se zabývá vývojem výukové webové aplikace pro demonstraci P-úplných problémů. Cílem je usnadnit studentům pochopení a procvičování těchto problémů. Aplikace se zaměřuje na tři P-úplné problémy: *Monotone Circuit Value Problem* (MCVP), *problém neprázdnosti bezkontextové gramatiky* a *kombinatorické hry na grafu*. Práce zahrnuje implementaci modulů pro interaktivní zadávání vstupů (ručně, náhodnou generací nebo výběrem z připravené sady) a simulaci jejich řešení. Uživatel může také sledovat krokový převod instance MCVP na další zmíněné problémy a následně jejich řešení.

## Klíčová slova

Teoretická informatika, P-úplné problémy, Monotone Circuit Value Problem, bezkontextová gramatika, kombinatorické hry, webová aplikace, simulace, převod instancí

## Abstract

This bachelor's thesis focuses on the development of a teaching-oriented web application for illustrating P-complete problems. The main goal is to facilitate students' understanding and practice of such tasks. The application focuses on the *Monotone Circuit Value Problem* (MCVP), *Context-Free Grammar Non-emptiness Problem*, and *Combinatorial Games on Graphs*. The project implements modules for interactive input of problem instances (manually, via random generation, or by selecting from a pre-defined set) and provides a simulation of their solutions. Users can also observe a step-by-step reduction from an MCVP instance to the other mentioned problems and subsequently explore how those are solved. The resulting application demonstrates key concepts of theoretical computer science, including the notion of P-completeness, and provides a flexible basis for educational use.

## Keywords

Theoretical computer science, P-complete problems, Monotone Circuit Value Problem, context-free grammar, combinatorial games, web application, simulation, instance reduction

# Obsah

**Seznam použitých symbolů a zkratek** **6**

**Seznam obrázků** **7**

**1 Úvod** **8**

**2 Teoretický základ** **10**
2.1 Základní pojmy a definice . . . . . 10
2.2 Monotone Circuit Value Problem . . . . . 10
2.3 Kombinatorické hry na grafu . . . . . 11
2.4 Neprázdnost bezkontextové gramatiky . . . . . 12
2.5 Principy převodů mezi problémy . . . . . 12

**3 Analýza a návrh** **14**
3.1 Volba technologií . . . . . 14
3.2 Architektura a struktura kódu . . . . . 14
3.3 Návrh datových struktur . . . . . 15
3.4 Návrh uživatelského rozhraní . . . . . 16

**4 Implementace** **18**
4.1 Monotone Circuit Value Problem . . . . . 18
4.2 Kombinatorická hra . . . . . 24
4.3 Bezkontextové gramatiky . . . . . 29
4.4 Implementace převodů mezi problémy . . . . . 36

**5 Závěr** **40**
5.1 Dosažené výsledky . . . . . 40
5.2 Vzdělávací přínos . . . . . 41
5.3 Možnosti dalšího rozvoje . . . . . 41

**Literatura** **42**

# Seznam použitých zkratek a symbolů

| | | |
|---|---|---|
| MCVP | – | Monotone Circuit Value Problem |
| AND | – | Logický součin – hradlo typu AND |
| OR | – | Logický součet – hradlo typu OR |
| P | – | Třída problémů řešitelných v polynomiálním čase |
| NC | – | Nick's Class – třída efektivně paralelizovatelných problémů |
| BFS | – | Breadth First Search – Algoritmus průchodu do šířky |
| CFG | – | Context-Free Grammar – Bezkontextová gramatika |
| DAG | – | Directed Acyclic Graph – Orientovaný acyklický graf |
| DFS | – | Depth First Search – Algoritmus průchodu do hloubky |
| DOM | – | Document Object Model – Objektový model dokumentu |
| ID | – | Identifier – Identifikátor |
| JSON | – | JavaScript Object Notation – Objektový zápis JavaScriptu |
| LS | – | Levá strana |
| PS | – | Pravá strana |
| SPA | – | Single Page Application – Jednostránková webová aplikace |

# Seznam obrázků


3.1 Třídní diagram pro hlavní třídy MCVP . . . . . 15
3.2 Třídní diagram pro hlavní třídy kombinatorické hry . . . . . 16
3.3 Třídní diagram pro hlavní třídu bezkontextové gramatiky . . . . . 16
3.4 Ovládací panel s navigací a výběrem vstupu . . . . . 17
3.5 Interaktivní plátno pro vizualizaci a editaci grafů . . . . . 17
3.6 Informační panel zobrazující výsledek analýzy . . . . . 17

4.1 Výběr způsobu zadání vstupu pro MCVP . . . . . 18
4.2 Interaktivní editor pro tvorbu a úpravu monotónních obvodů . . . . . 22
4.3 Ukázka JSON formátu pro uložení MCVP obvodu . . . . . 23
4.4 Vizualizace MCVP obvodu pomocí TreeRenderCanvas . . . . . 24
4.5 Ukázka předpřipravených herních grafů ve formátu JSON . . . . . 26
4.6 Ovládací prvky pro import a export herních grafů . . . . . 26
4.7 Kroková analýza kombinatorické hry . . . . . 28
4.8 Rozhraní pro nastavení parametrů generování bezkontextových gramatik . . . . . 31
4.9 Ukázka formátu JSON pro uložení bezkontextové gramatiky . . . . . 33
4.10 Vizualizace derivačního stromu pro neprázdnou gramatiku . . . . . 34
4.11 Kroková analýza bezkontextové gramatiky . . . . . 35
4.12 Ukázka převodu MCVP obvodu na kombinatorickou hru . . . . . 37
4.13 Ukázka převodu MCVP obvodu na bezkontextovou gramatiku . . . . . 39

# Kapitola 1

# Úvod

Teoretická informatika poskytuje způsob, jak zkoumat, porozumět a optimalizovat možnosti a limity výpočetních systémů. Hlavním bodem tohoto zkoumání je teorie složitosti, která klasifikuje problémy na základě zdrojů potřebných pro jejich vyřešení, jako je čas a paměť. Jednou z nejvýznamnějších složitostních tříd je třída $P$, zahrnující problémy řešitelné v polynomiálním čase na deterministickém Turingově stroji [10] – tedy takové, jejichž časovou složitost lze vyjádřit jako $O(n^k)$ [11] pro nějakou konstantu $k$, kde $n$ je velikost vstupu. Přestože jsou problémy v této třídě obvykle pokládány za „efektivně řešitelné“, projevují se mezi nimi významné odlišnosti, zejména pokud začneme řešit jejich paralelizovatelnost.

V tomto kontextu hraje klíčovou roli podmnožina *P-úplných* problémů (*P-complete* problems). P-úplné problémy představují výpočetně nejnáročnější úlohy v rámci třídy $P$. Jsou to problémy, na které lze s logaritmickou paměťovou složitostí převést jakýkoliv jiný problém z třídy $P$ [7, 1]. Hlavní problém u této podmnožiny spočívá v předpokladu, že P-úplné problémy pravděpodobně nelze efektivně paralelizovat. To znamená, že tyto úlohy nespadají do třídy $NC$, která obsahuje problémy řešitelné v polylogaritmickém čase $O(\log^k n)$ pomocí paralelního výpočtu s polynomiálně mnoha procesory [8]. Z toho vyplývá, že na rozdíl od NC problémů řešení P-úplných problémů vyžaduje sekvenční zpracování. Studium P-úplnosti nám tak pomáhá vymezit hranici mezi paralelizovatelným a čistě sekvenčním výpočtem.

Tento projekt představuje interaktivní ukázku konceptu P-úplnosti prostřednictvím webové aplikace. Cílem je vytvořit nástroj, který umožní uživatelům vizualizovat a pochopit výpočet a převod mezi zvolenými P-úplnými problémy.

Jako hlavní problém byl zvolen *Monotone Circuit Value Problem* (MCVP), ve kterém se vyhodnocuje logický obvod tvořený pouze hradly typu AND a OR [8]. Pro demonstraci univerzálnosti konceptu P-úplnosti aplikace implementuje také simulace dvou dalších problémů:

- **Kombinatorické hry na grafu:** Úloha analyzující existenci vítězné strategie v deterministické hře dvou hráčů [8].

- **Problém neprázdnosti jazyka bezkontextové gramatiky:** Rozhoduje, zda daná gramatika generuje neprázdný jazyk [8].

Hlavním přínosem vytvořené aplikace je možnost vizualizace nejen samotného řešení těchto úloh, ale především *převodů* (redukcí) mezi nimi. Uživatel může sledovat krokovou transformaci instance MCVP na instanci hry nebo gramatiky, což názorně ilustruje princip polynomiálních redukcí a vzájemnou převoditelnost P-úplných problémů [8].

Výsledkem práce je webová aplikace navržená jako flexibilní nástroj pro výuku. Umožňuje uživatelům pracovat s vlastními vstupy, generovat náhodné instance pro testování a využívat předpřipravené sady úloh.

Text práce je členěn do několika částí. Po úvodu následuje kapitola věnovaná teoretickému základu, definicím klíčových pojmů a principům redukcí. Další část se zabývá analýzou a návrhem aplikace, volbou technologií a popisem datových struktur. Stěžejní částí práce je kapitola věnovaná detailnímu popisu implementace všech tří problémů a jejich vzájemných převodů. Závěr práce shrnuje dosažené výsledky a navrhuje možnosti dalšího rozšíření.

# Kapitola 2

# Teoretický základ

Tato kapitola zavádí základní pojmy z teorie složitosti a popisuje tři P-úplné problémy: Monotone Circuit Value Problem, kombinatorické hry na grafu a problém neprázdnosti bezkontextových gramatik.

## 2.1 Základní pojmy a definice

Nejprve jsou zavedeny klíčové pojmy z teorie složitosti, o které se tato práce opírá.

**Definice 1 (P-úplnost [8, 7])** *Problém $A$ se nazývá **P-úplný**, jestliže platí dvě podmínky:*

1. *$A \in P$ (problém patří do třídy $P$).*
2. *Pro každý problém $B \in P$ platí $B \leq_L A$ (každý problém z třídy $P$ lze na problém $A$ převést pomocí redukce v logaritmickém prostoru).*

**Definice 2 (Třída P [10])** *Třída **P** (Polynomial time) obsahuje všechny rozhodovací problémy, které jsou řešitelné na deterministickém Turingově stroji v čase $O(n^k)$, kde $n$ je velikost vstupu a $k$ je nezáporná konstanta.*

**Definice 3 (Logaritmická redukce [8])** *Nechť $A$ a $B$ jsou jazyky (problémy). Řekneme, že $A$ je **převeditelný v logaritmickém prostoru** na $B$ (značíme $A \leq_L B$), jestliže existuje funkce $f$ vyčíslitelná Turingovým strojem s logaritmickou paměťovou složitostí taková, že pro každé slovo $w$ platí:*

$$w \in A \iff f(w) \in B$$

## 2.2 Monotone Circuit Value Problem

Monotone Circuit Value Problem je základní problém v teorii složitosti, který patří mezi P-úplné problémy [8]. Jeho definice je následující:

- **Vstup:** Booleovský obvod bez negací, tvořený pouze hradly AND ($\wedge$) a OR ($\vee$), společně se vstupními hodnotami (pouze hodnoty 0 nebo 1 - false nebo true).
- **Výstup:** Hodnota výstupního uzlu obvodu (výstupního hradla).

Obvod lze reprezentovat jako orientovaný acyklický graf (DAG), kde uzly představují buď vstupní proměnné (listy) nebo logická hradla (vnitřní uzly). Hrany reprezentují tok logických hodnot – přenášejí výsledky vyhodnocení z jednoho uzlu jako vstupy do uzlů následujících. V monotónním obvodu chybí hradla NOT, což zajišťuje, že funkce reprezentovaná obvodem je monotónní – zvýšení hodnoty libovolného vstupu nikdy nesníží hodnotu výstupu [8].

### 2.2.1 P-úplnost MCVP

MCVP je P-úplný problém [8, 6]. Vyhodnocení monotónního obvodu lze provést v polynomiálním čase postupným vyhodnocováním uzlů od vstupů směrem k výstupu. MCVP je sice řešitelný v polynomiálním čase, ale jeho P-úplnost znamená, že efektivní paralelizace je pravděpodobně nemožná – vyhodnocení obvodu musí probíhat sekvenčně od vstupů k výstupu.

Pro P-úplnost MCVP je zásadní, že obvod je reprezentován jako DAG, nikoliv jako strom. Tentýž logický výraz může být jako DAG výrazně menší než odpovídající strom, protože v DAGu se stejné podformule vyskytují pouze jednou, zatímco ve stromě tolikrát, kolikrát je jejich hodnota dále využívána. Velikost vstupu $n$ je tedy u DAGu menší a při složitosti $O(n^k)$ je k dispozici méně kroků na provedení výpočtu. Dosáhnout polynomiální složitosti je tak na DAGu těžší než na stromě – problém vyhodnocení na stromě je sice v P, ale není P-těžký (a tedy ani P-úplný).

## 2.3 Kombinatorické hry na grafu

Kombinatorická hra dvou hráčů na orientovaném grafu je další příklad P-úplného problému [8]:

- **Vstup:** Orientovaný graf, kde každý uzel představuje herní pozici přiřazenou Hráči 1 nebo Hráči 2, hrany reprezentují možné tahy a jeden uzel je označen jako startovní pozice.
- **Výstup:** Rozhodnutí, zda Hráč 1 má výherní strategii ze startovní pozice.

Hráči se střídají v tazích podle toho, komu je přiřazen aktuální uzel. Hra končí, když hráč na tahu nemá žádný možný tah – tento hráč pak prohrává. Výherní strategie pro Hráče 1 znamená, že existuje způsob volit tahy tak, aby Hráč 1 vyhrál bez ohledu na to, jak hraje Hráč 2 [8].

### 2.3.1 P-úplnost kombinatorických her

Tento problém je P-úplný [8, 6]. Určení výherní strategie lze provést v polynomiálním čase pomocí tzv. retrográdní analýzy [8], která zpětně vyhodnocuje pozice od koncových uzlů. P-úplnost pro-

blému implikuje obtížnost paralelizace: výherní strategie nelze určit efektivně paralelně, protože analýza pozic na sebe vzájemně závisí.

## 2.4 Neprázdnost bezkontextové gramatiky

Problém neprázdnosti jazyka bezkontextové gramatiky (CFG Non-emptiness Problem) je dalším příkladem P-úplného problému [8]. Bezkontextová gramatika generuje jazyk pomocí přepisovacích pravidel.

Formálně je gramatika definována jako čtveřice $G = (N, \Sigma, P, S)$ [9], kde:

- $N$ je konečná množina neterminálních symbolů (neterminálů).
- $\Sigma$ je konečná množina terminálních symbolů (terminálů), disjunktní s $N$.
- $P$ je konečná množina přepisovacích pravidel tvaru $A \to \alpha$, kde $A \in N$ a $\alpha \in (N \cup \Sigma)^*$.
- $S \in N$ je počáteční symbol (start symbol).

Problém spočívá v tom, zda daná gramatika generuje alespoň jedno slovo složené pouze z terminálních symbolů.

### 2.4.1 P-úplnost problému neprázdnosti

Problém neprázdnosti jazyka bezkontextové gramatiky je P-úplný [8, 6]. Tento problém lze vyřešit v polynomiálním čase pomocí algoritmu iterativního označování produktivních neterminálů. Podobně jako u předchozích problémů, i zde P-úplnost vyplývá ze sekvenčních závislostí: produktivita neterminálu závisí na produktivitě jiných neterminálů, což brání paralelnímu zpracování.

## 2.5 Principy převodů mezi problémy

P-úplné problémy jsou vzájemně převoditelné, což dokazuje jejich výpočetní rovnocennost. Následující převody ilustrují, jak lze MCVP převést na kombinatorickou hru a na problém neprázdnosti gramatiky.

### 2.5.1 Převod z MCVP na kombinatorickou hru

Uzly obvodu MCVP odpovídají herním pozicím [8]:

- **Hradlo OR** vytváří pozici pro Hráče 1. Ten si může vybrat libovolnou následující pozici odpovídající potomkům hradla. Stačí, aby jedna z možností vedla k jeho výhře.

- **Hradlo AND** vytváří pozici pro Hráče 2. Ten vybírá následující pozici a snaží se zabránit výhře Hráče 1. Hráč 1 vyhraje pouze pokud vyhrává ve všech možných pokračováních.
- **Proměnná s hodnotou 1** odpovídá konečné pozici, ve které Hráč 2 nemá žádné tahy – Hráč 1 tedy vyhrává.
- **Proměnná s hodnotou 0** odpovídá konečné pozici, ve které Hráč 1 nemá žádné tahy a prohrává.

### 2.5.2 Převod z MCVP na bezkontextovou gramatiku

Každý uzel obvodu se stane symbolem gramatiky [8]:

- **Kořen obvodu** se mapuje na počáteční symbol gramatiky $S$.
- **Hradlo AND s potomky** $A$ **a** $B$ vytvoří pravidlo $X \to AB$, kde $X$ reprezentuje hradlo. Řetězec lze z $X$ odvodit právě tehdy, když lze odvodit řetězce z obou potomků $A$ i $B$.
- **Hradlo OR s potomky** $A$ **a** $B$ vytvoří dvě pravidla: $X \to A$ a $X \to B$.
- **Proměnná s hodnotou 1** vytvoří pravidlo $X \to t$, kde $t$ je terminál.
- **Proměnná s hodnotou 0** generuje epsilon pravidlo $X \to \varepsilon$, což umožňuje odvození prázdného řetězce.

# Kapitola 3

# Analýza a návrh

Tato kapitola popisuje zvolené technologie a architekturu aplikace. Dále představuje datové struktury pro reprezentaci jednotlivých problémů a principy návrhu uživatelského rozhraní.

## 3.1 Volba technologií

Aplikace byla implementována jako webová, protože nevyžaduje instalaci a běží v libovolném moderním prohlížeči. Webové řešení také umožňuje snadnou údržbu a okamžitou distribuci aktualizací.

### 3.1.1 React a Vite

Pro vývoj byla použita knihovna *React* [5]. React umožňuje rozdělit uživatelské rozhraní na znovupoužitelné komponenty, což usnadnilo vývoj modulů pro jednotlivé problémy. Díky reaktivitě se rozhraní automaticky synchronizuje se stavem dat bez nutnosti ruční manipulace s DOM.

Pro sestavení aplikace byl použit nástroj *Vite* [12]. Vite nabízí rychlé vývojové prostředí s okamžitou odezvou při úpravách kódu a optimalizuje soubory pro produkční verzi.

## 3.2 Architektura a struktura kódu

Aplikace je navržena jako *Single Page Application* (SPA) – veškerá logika a navigace probíhá v rámci jedné stránky, podobně jako u desktopové aplikace. Kořenová komponenta `App.jsx` spravuje globální stav a přepínání mezi moduly.

Kód je strukturován tak, aby logika byla oddělena od vizualizace. Každý modul (MCVP, hry, gramatiky) disponuje vlastní adresářovou strukturou:

- **Hlavní komponenta:** Funguje jako kontejner, který spravuje lokální stav a propojuje vstupní data s algoritmy a vizualizací.

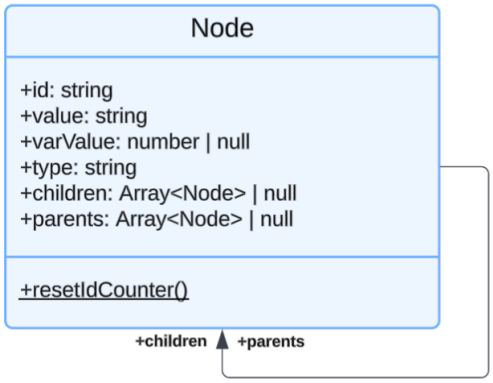
- **Adresář Utils:** Zde jsou soustředěny čisté algoritmy – parsery pro zpracování vstupu, generátory náhodných instancí a evaluátory pro výpočet řešení.
- **Vizualizační komponenty:** Starají se o vykreslení grafů a stromů pomocí knihovny *react-force-graph*.

## 3.3 Návrh datových struktur

Pro práci s P-úplnými problémy bylo nutné navrhnout jejich vnitřní reprezentaci v paměti.

### 3.3.1 Reprezentace obvodu MCVP

Pro modelování logického obvodu byl využit orientovaný acyklický graf (DAG). Každý uzel obvodu je reprezentován instancí třídy `Node`. Tato třída, jak ukazuje třídní diagram na obrázku 3.1, uchovává informaci o typu operace (AND, OR) nebo hodnotě proměnné. Seznam odkazů na potomky umožňuje průchod grafem při vyhodnocování.




Obrázek 3.1: Třídní diagram pro hlavní třídy MCVP

### 3.3.2 Reprezentace herního grafu

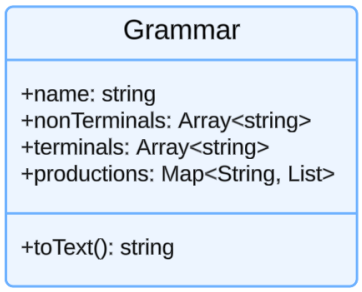
Na rozdíl od MCVP může herní graf obsahovat cykly, proto byly navrženy třídy `GamePosition` a `GameGraph` (viz obrázek 3.2). Třída `GamePosition` reprezentuje jednotlivou pozici a uchovává informaci o hráči na tahu spolu se seznamy předchůdců a následovníků, což je nezbytné pro retrográdní analýzu.

### 3.3.3 Reprezentace bezkontextové gramatiky

U bezkontextových gramatik je zaměřeno na reprezentaci přepisovacích pravidel. Třída `Grammar` (obrázek 3.3) spravuje množiny symbolů a seznamy produkcí. Každá produkce propojuje levou

Obrázek 3.2: Třídní diagram pro hlavní třídy kombinatorické hry

stranu (neterminál) s polem symbolů na pravé straně. Tato struktura umožňuje snadno identifikovat produktivní neterminály.




Obrázek 3.3: Třídní diagram pro hlavní třídu bezkontextové gramatiky

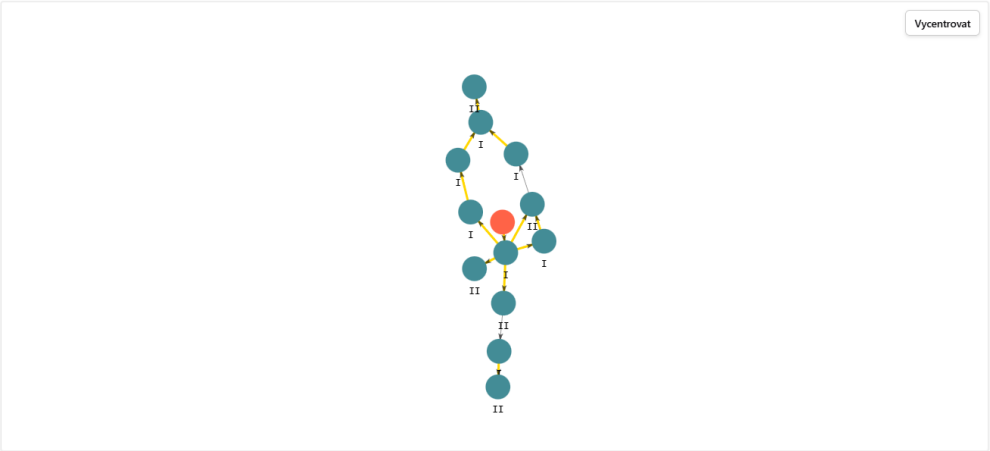
## 3.4 Návrh uživatelského rozhraní

Rozhraní je navrženo tak, aby student nebyl zahlcen informacemi a mohl se soustředit na podstatu problému. Rozložení stránky bylo sjednoceno napříč všemi moduly:

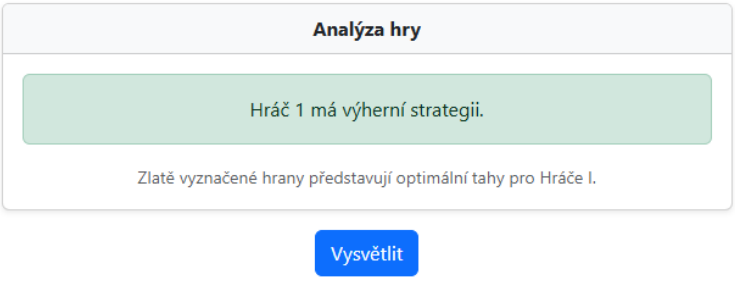
- **Ovládací panel:** Horní část obrazovky, kde uživatel volí způsob zadání vstupu a spouští simulace (viz obrázek 3.4).
- **Interaktivní plátno:** Hlavní plocha pro vizualizaci grafů, kde uživatel může s prvky přímo manipulovat (viz obrázek 3.5).
- **Informační panely:** Modální okna nebo postranní panely zobrazující detaily o probíhajícím výpočtu nebo krokovém převodu (viz obrázek 3.6).

Pro vizualizaci byla použita knihovna *react-force-graph* [2], která pomocí fyzikální simulace automaticky rozmísťuje uzly a hrany. Stylování je realizováno frameworkem *Bootstrap 5* [3], který zajišťuje jednotný vzhled a responzivitu na různých velikostech obrazovek.

Obrázek 3.4: Ovládací panel s navigací a výběrem vstupu




Obrázek 3.5: Interaktivní plátno pro vizualizaci a editaci grafů




Obrázek 3.6: Informační panel zobrazující výsledek analýzy

# Kapitola 4

# Implementace

Tato kapitola detailně popisuje implementaci jednotlivých modulů aplikace. Zaměřuje se na technické řešení algoritmů pro parsování, vyhodnocování a generování instancí P-úplných problémů.

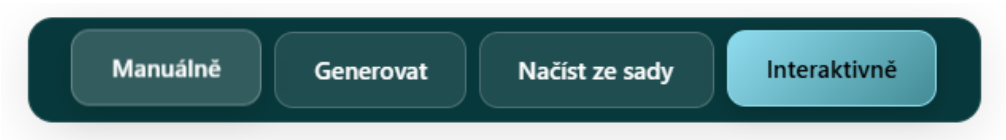
## 4.1 Monotone Circuit Value Problem

Modul pro MCVP tvoří jádro aplikace a slouží jako výchozí bod pro převody na ostatní problémy.

### 4.1.1 Formát vstupu

Aplikace nabízí čtyři způsoby zadávání monotónního obvodu (viz obrázek 4.1):

- **Textový zápis:** Zadání pomocí logického výrazu, který je následně parsován (viz sekce 4.1.1.1).
- **Interaktivní editace:** Vytváření obvodu přímo na plátně pomocí grafického editoru (viz sekce 4.1.3).
- **Generování náhodných obvodů:** Automatické vytvoření obvodu dle parametrů (viz sekce 4.1.4).
- **Předpřipravené sady:** Načtení ukázkových obvodů z připravené knihovny (viz sekce 4.1.5).




Obrázek 4.1: Výběr způsobu zadání vstupu pro MCVP

#### 4.1.1.1 Manuální zadání a parsování výrazů

Textový zápis ve formě logického výrazu je první metodou zadání vstupu. Tento zápis je následně převeden na strukturu obvodu (DAG).

**4.1.1.1.1 Gramatika vstupního jazyka** Vstupní výrazy odpovídají jednoduché gramatice popsané v kapitole 2.2. Pro připomenutí:

- Operátory `O` (OR) a `A` (AND) reprezentují logický součet a součin.
- Proměnné jsou zapsány ve tvaru `x1[0]` nebo `x2[1]`, kde číslo v hranatých závorkách udává hodnotu proměnné (0 nebo 1).
- Výrazy lze libovolně uzávorkovat pomocí kulatých závorek pro určení priority vyhodnocení.

Příklad platného výrazu: `((x1[1] A x2[0]) O (x3[1] A x4[1]))`

#### 4.1.1.2 Lexikální a syntaktická analýza

Převod textového výrazu na grafovou strukturu obvodu (DAG) probíhá ve dvou fázích, které jsou implementovány v modulu `Parser.js`:

1. **Tokenizace (lexikální analýza):** Funkce `tokenize()` rozdělí vstupní řetězec na posloupnost tokenů. Každý token je dvojice (typ, hodnota), například `['LPAREN', '(']` nebo `['VARIABLE', 'x1[1]']`. Tokenizér rozpoznává závorky, operátory (A, O) a proměnné pomocí regulárních výrazů.
2. **Parsování (syntaktická analýza):** Třída `Parser` implementuje rekurzivní sestupný parser respektující prioritu operátorů. Výrazy s operátorem OR mají nejnižší prioritu, následují výrazy s operátorem AND a nejvyšší prioritu mají atomické výrazy v závorkách nebo proměnné. Parser vytváří uzly typu `Node` (viz třídní diagram v sekci 3.3.1, obrázek 3.1), které tvoří strukturu grafu reprezentující zadaný obvod.

Parser současně kontroluje výskyty opakovaných proměnných. Pokud se stejná proměnná v jednom vstupním výrazu objeví vícekrát, používá se v interní reprezentaci již vytvořený uzel a je vyžadována stejná hodnota, jak byla přiřazena v prvním výskytu. Při konfliktu (například `x1[0]` a `x1[1]`) je vstup vyhodnocen jako neplatný a uživateli je vrácena chybová hláška.

Výsledkem parsování je orientovaný graf, kde vstupní uzly jsou proměnné s přiřazenými hodnotami a vnitřní uzly reprezentují logické operace.

### 4.1.2 Vyhodnocení obvodu

Vyhodnocení monotónního obvodu probíhá rekurzivním průchodem grafu, který je implementován v modulu `EvaluateCircuit.js`.

#### 4.1.2.1 Algoritmus vyhodnocení

Funkce `evaluateCircuitWithSteps()` používá algoritmus průchodu do hloubky (DFS) s memoizací. Pro maximální využití vypočítaných dat funkce vrací jak výsledek vyhodnocení, tak i sekvenci všech mezivýpočtů, které můžou být použity pro krokovou vizualizaci. Průběh algoritmu:

1. **Listy (proměnné):** Pro uzly typu `variable` funkce vrací přímo přiřazenou hodnotu (0 nebo 1).

2. **Vnitřní uzly (operace):** Pro uzly reprezentující logické operace funkce nejdříve vyhodnotí všechny potomky. Pokud je uzel typu AND, výsledek je 1 právě tehdy, když všichni potomci mají hodnotu 1. Pokud je uzel typu OR, výsledek je 1 pouze tehdy, když alespoň jeden potomek má hodnotu 1.

3. **Zaznamenávání kroků:** Po vyhodnocení každého vnitřního uzlu je zaznamenán objekt obsahující vyhodnocovaný uzel, hodnoty jeho potomků a vypočítaný výsledek.

4. **Memoizace:** Vyhodnocené hodnoty uzlů jsou uloženy do cache (slovníku), aby nedocházelo k opakovanému výpočtu stejných uzlů.

5. **Detekce cyklů:** Při vstupu do uzlu je tento přidán do množiny `visiting`. Pokud algoritmus narazí na uzel, který je již v této množině, detekuje cyklus a vyhodí chybu. Po dokončení vyhodnocení uzlu je uzel z množiny `visiting` odstraněn.

Časová složitost algoritmu je $O(n)$, kde $n$ je počet uzlů v obvodu, protože každý uzel je vyhodnocen právě jednou díky memoizaci. Funkce vrací objekt obsahující samotný výsledek a pole všech kroků výpočtu.

#### 4.1.2.2 Krokové vyhodnocení

Vyhodnocení obvodu je provedeno pouze jednou při načtení, výsledek ve formě výstupní hodnoty obvodu je využit pro běžné zobrazení a zaznamenané kroky jsou předány modální komponentě `StepByStepTree` pro krokovou vizualizaci. Tím se předchází opakovanému průchodu grafem a duplicitnímu výpočtu, což je výhodné zejména u rozsáhlejších obvodů.

Komponenta `StepByStepTree` zobrazuje každý krok graficky – aktuálně vyhodnocovaný uzel je zvýrazněn a pod grafem je zobrazena informace o hodnotách vstupů a výsledku operace. Uživatel tak může sledovat, jak se hodnoty šíří od vstupů k výstupu obvodu a lépe pochopit princip vyhodnocování.

Stejně jako u dalších modálních vizualizací se i zde zachovávají uživatelem upravené pozice uzlů z hlavního zobrazení.

### 4.1.3 Interaktivní editace obvodu

Kromě textového zadání umožňuje aplikace vytvářet a upravovat obvody pomocí grafického editoru implementovaného v komponentě `InteractiveMCVPGraph` (viz obrázek 4.2). Uživatel může:

- **Přidávat uzly:** Pomocí tlačítek pod grafem lze vložit nové uzly typu AND, OR nebo vstupní proměnnou s hodnotou 0 nebo 1. Nový uzel se umístí do grafu, ale není propojen s ostatními uzly.
- **Vytvářet hrany:** Označit uzel, ten následně může:
  - Nastavit na AND nebo OR operaci.
  - Nastavit na proměnnou s hodnotou 0 nebo 1.
  - Odstranit uzel z grafu.
  - Vytvořit nebo odstranit hranu s dalším uzlem.
- **Reorganizovat graf:** Uzly lze přesouvat myší pro lepší vizuální uspořádání obvodu.

Při vytváření hran je v editoru vynuceno binární omezení hradel: uzly operací (AND a OR) mohou mít nejvýše dva potomky.

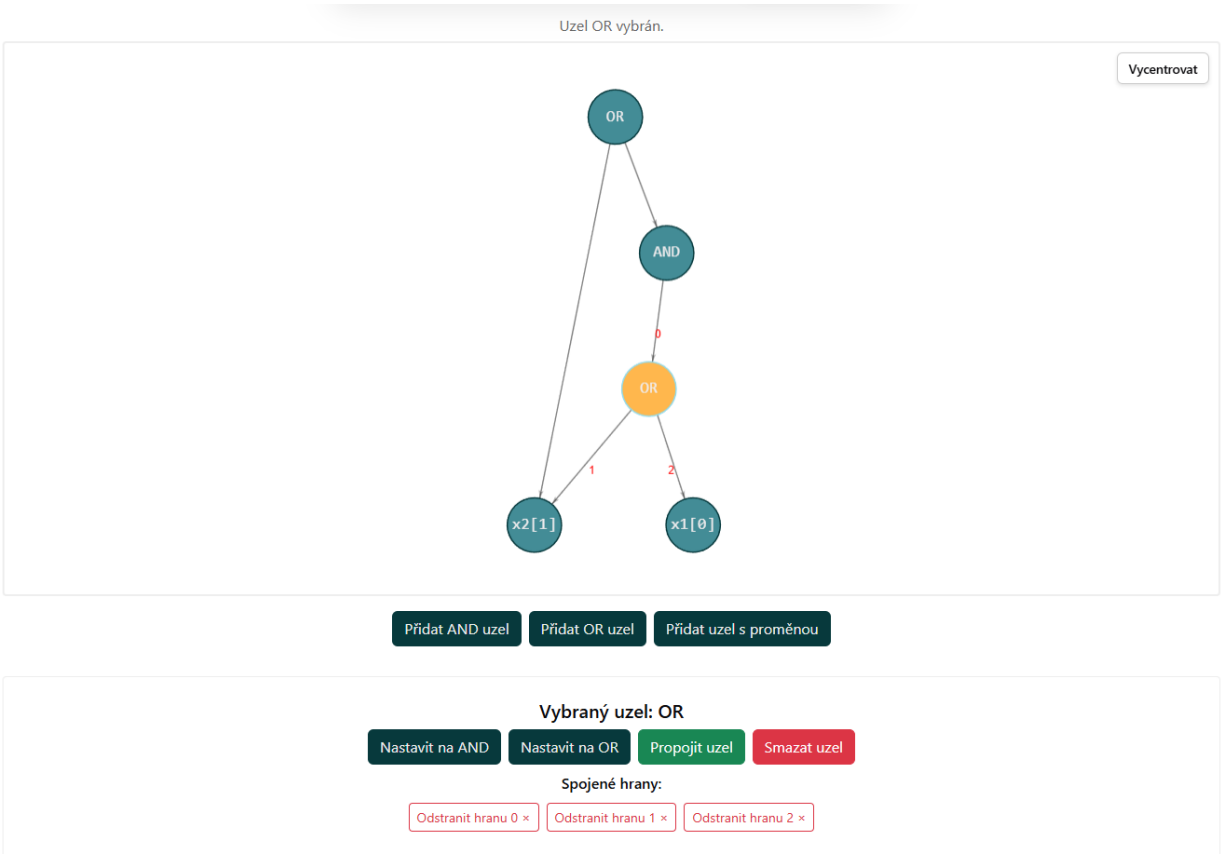
### 4.1.4 Generování náhodných obvodů

Aplikace obsahuje generátor náhodných obvodů v modulu `Generator.js`. Uživatel může upravit dva parametry:

- **Počet hradel:** Kolik uzlů s operacemi (hradel AND a OR) bude obvod obsahovat.
- **Počet proměnných:** Kolik uzlů s proměnnými (listů) bude obvod obsahovat.

Generování probíhá v následujících krocích:

1. Algoritmus vytvoří zadaný počet proměnných s náhodnou hodnotou 0 nebo 1.
2. Algoritmus postupně přidává hradla. Pro každé nové hradlo vypočítá cílový počet vstupních uzlů (potomků) podle vzorce $\lceil n/k \rceil$, kde $n$ je počet dosud vytvořených uzlů v grafu a $k$ je počet zbývajících hradel, která je třeba vytvořit.
3. K vypočítané hodnotě se přidá náhodná odchylka $\pm 20\%$ – tak je dosaženo vyváženosti mezi úplnou náhodností a vyrovnanou distribucí.
4. Hradlo náhodně vybere určený počet uzlů jako potomky a náhodně zvolí operaci (AND nebo OR).




Obrázek 4.2: Interaktivní editor pro tvorbu a úpravu monotónních obvodů

5. Nově vytvořené hradlo se přidá do množiny použitelných uzlů pro následující hradla.

6. Poslední hradlo využije jako potomky všechny uzly, které dosud nebyly vybrány jako potomci jiného hradla, a stane se kořenem obvodu.

Generátor upřednostňuje binární zapojení: při výběru potomků se pokud možno vytváří dvojice vstupů tak, aby hradla měla dva potomky a současně byl zachován DAG vlastnosti grafu. Generátor navíc může opětovně použít již vytvořených uzlů jako vstupů pro více hradel, takže výsledné instance nejsou omezeny pouze na stromové struktury, ale obsahují i sdílené podgrafy.

Výsledný graf je platný DAG.

### 4.1.5 Připravené sady obvodů

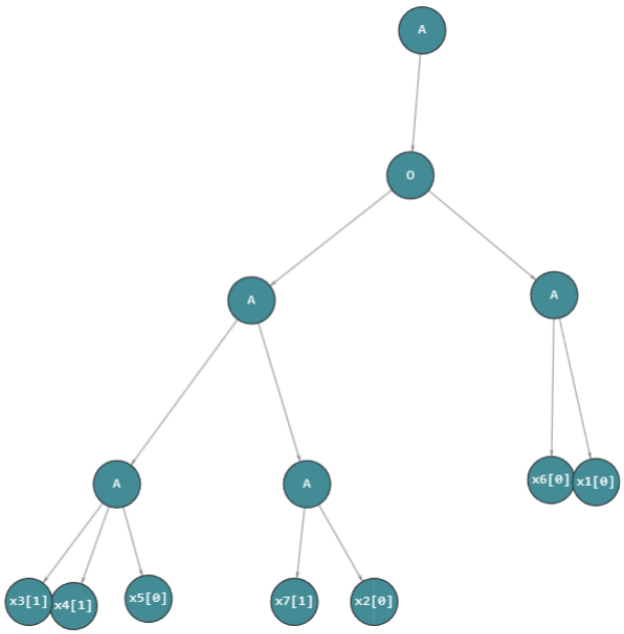
Aplikace obsahuje sadu připravených obvodů, které lze načíst pomocí komponenty `PreparedSetsInput`. Tyto obvody jsou uloženy ve složce `Sady/MCVP` ve formátu JSON, který obsahuje pole `nodes` s uzly a pole `edges` s hranami. Každý uzel má atributy `id`, `type` (AND, OR, variable), a pro proměnné také `value` (0 nebo 1). Hrany jsou reprezentovány jako dvojice `source` a `target`, které odkazují na ID uzlů.

```
1  {
2    "name": "Sada 1 (4x Proměnné, 3x Operace)",
3    "nodes": [
4      { "id": 1, "type": "variable", "value": "x1", "varValue": 0 },
5      { "id": 2, "type": "variable", "value": "x2", "varValue": 1 },
6      { "id": 3, "type": "variable", "value": "x3", "varValue": 1 },
7      { "id": 4, "type": "variable", "value": "x4", "varValue": 0 },
8      { "id": 5, "type": "operation", "value": "O" },
9      { "id": 6, "type": "operation", "value": "A" },
10     { "id": 7, "type": "operation", "value": "A" }
11   ],
12   "edges": [
13     { "source": 5, "target": 1 },
14     { "source": 5, "target": 2 },
15     { "source": 6, "target": 3 },
16     { "source": 6, "target": 4 },
17     { "source": 7, "target": 5 },
18     { "source": 7, "target": 6 }
19   ]
20 }
```

Obrázek 4.3: Ukázka JSON formátu pro uložení MCVP obvodu

### 4.1.6 Vizualizace obvodu

Vizualizace monotónního obvodu je implementována v komponentě `TreeRenderCanvas`. Obvod je zobrazen jako orientovaný graf s výstupním hradlem nahoře a vstupy dole, hodnoty se tedy šíří směrem zdola nahoru (viz obrázek 4.4).




Obrázek 4.4: Vizualizace MCVP obvodu pomocí TreeRenderCanvas

#### 4.1.6.1 Hierarchické rozložení

Uzly jsou automaticky umístěny do vrstev podle jejich vzdálenosti od kořene díky režimu `dagMode="td"` (top-down neboli shora dolů). Rozložení kombinuje hierarchickou strukturu s fyzikálními silami pro optimální vizuální uspořádání grafu.

## 4.2 Kombinatorická hra

### 4.2.1 Formát vstupu

Aplikace nabízí tři způsoby zadávání herního grafu:

- **Interaktivní editace:** Uživatel vytváří a upravuje graf pomocí grafického editoru (viz sekce 4.2.2).
- **Generování náhodných her:** Automatické vytvoření náhodného grafu podle zadaných parametrů (viz sekce 4.2.3).
- **Předpřipravené sady:** Načtení ukázkových připravených příkladů (viz sekce 4.2.4).

Všechny formy vstupu využívají komponentu `DisplayGraph` pro vizualizaci herního grafu (viz sekce 4.2.5).

### 4.2.2 Interaktivní editace grafu

Komponenta `ManualInput` umožňuje vytvářet a upravovat herní grafy pomocí interaktivního grafického editoru. Uživatel může:

- **Přidávat uzly:** Vytvořit novou pozici.
- **Upravovat uzly:** Po označení uzlu lze:
  - Změnit hráče, který je na tahu (Hráč 1 nebo Hráč 2)
  - Odstranit pozici z grafu
  - Nastavit jako počáteční pozici
  - Vytvořit hranu z tohoto uzlu
  - Odstranit hrany k nebo od tohoto uzlu
- **Reorganizovat graf:** Uzly lze přesouvat myší pro lepší vizuální uspořádání.

Aplikace průběžně validuje graf. Pokud není nastavena počáteční pozice, analýza není provedena a uživateli se zobrazí informační zpráva.

### 4.2.3 Generování náhodných her

Modul `Generator.js` obsahuje funkci `generateGraph()` pro vytváření náhodných herních grafů. Uživatel nastavuje dva parametry:

- **Počet pozic:** Kolik uzlů (herních pozic) bude graf obsahovat.
- **Pravděpodobnost hrany:** Hodnota 0%–100% určující, jak pravděpodobné je vytvoření hrany mezi dvěma uzly.

Algoritmus generování nejprve vytváří kostru grafu, která zajišťuje, že všechny pozice jsou dosažitelné z počáteční pozice, a následně přidává náhodné hrany, které mohou vytvářet cykly. Postup s $n$ pozicemi a indexy $i, j$ jednotlivých pozic:

1. **Vytvoření kostry:** Vytvoří se uzly očíslované 0 až $n - 1$ s náhodně přiřazeným hráčem. Pro každý uzel $i > 0$ vznikne hrana z náhodného předchozího uzlu do uzlu $i$. Tím je zaručena dosažitelnost všech uzlů z uzlu 0 (počáteční pozice).
2. **Přidání dalších hran:** Pro každou dvojici uzlů $i, j$ (kde $i \neq j$) se podle zadané pravděpodobnosti přidá hrana $i \to j$, pokud ještě neexistuje. Tyto hrany mohou vytvářet cykly a zvyšují složitost hry.

Vnitřní reprezentace grafu pomocí tříd `GamePosition` a `GameGraph` je popsána v sekci 3.3.2. Diagram těchto tříd je uveden na obrázku 3.2.

### 4.2.4 Předpřipravené sady

Ve složce `Sady/CombinatorialGame` najdeme předpřipravené herní grafy různé velikosti a složitosti.

```
1  {
2    "name": "Sada 2 (5 uzlů, 7 hran)",
3    "nodes": [
4      { "id": "1", "player": 1 },
5      { "id": "2", "player": 2 },
6      { "id": "3", "player": 1 },
7      { "id": "4", "player": 2 },
8      { "id": "5", "player": 1 }
9    ],
10   "edges": [
11     { "source": "1", "target": "2" },
12     { "source": "1", "target": "3" },
13     { "source": "2", "target": "4" },
14     { "source": "3", "target": "4" },
15     { "source": "4", "target": "5" },
16     { "source": "2", "target": "3" },
17     { "source": "3", "target": "5" }
18   ],
19   "startingPosition": "3"
20 }
```

Obrázek 4.5: Ukázka předpřipravených herních grafů ve formátu JSON

#### 4.2.4.1 Ukládání a načítání her

Herní grafy lze exportovat a importovat ve formátu JSON pomocí komponenty `FileTransferControls` (viz obrázek 4.6). Formát obsahuje pole `nodes` s uzly, pole `edges` s hranami a identifikátor `startingPosition` (viz obrázek 4.5).

Obrázek 4.6: Ovládací prvky pro import a export herních grafů

### 4.2.5 Vizualizace grafu

Vizualizace herního grafu využívá komponentu `DisplayGraph`. Graf zobrazuje:

- **Uzly:** Každý uzel reprezentuje herní pozici s označením hráče na tahu (1 nebo 2).
- **Počáteční pozice:** Označena oranžovou barvou.
- **Hrany:** Možné tahy jsou zobrazeny jako směrované hrany. Hrany patřící do výherní strategie jsou zvýrazněny tlustší čarou a žlutou barvou.

### 4.2.6 Algoritmus analýzy

Řešení problému kombinatorických her je implementováno v modulu `ComputeWinner.js`. Algoritmus využívá retrográdní analýzu založenou na principech uvedených v sekci 2.3.

#### 4.2.6.1 Vyhodnocení výherních pozic

Algoritmus určuje, zda je daná pozice výherní pro hráče, který je v ní na tahu, nebo zda skončí remízou. Vyhodnocení probíhá následovně:

1. **Koncové pozice:** Pozice bez dalších tahů jsou prohrávající pro hráče, který je v nich na tahu.
2. **Zpětné šíření:** Od koncových pozic se postupně propagují výsledky:
   - Pokud z aktuální pozice existuje tah do pozice, kde je na tahu soupeř a ta pozice je pro něj prohrávající, pak je aktuální pozice vyhrávající.
   - Pokud z aktuální pozice existuje tah do pozice, kde zůstává tentýž hráč na tahu a ta pozice je vyhrávající, pak je i aktuální pozice vyhrávající.
   - Pokud všechny tahy z aktuální pozice vedou do pozic, které jsou vyhrávající pro soupeře (nebo prohrávající pro aktuálního hráče při zachování tahu), je aktuální pozice prohrávající.
3. **Neurčené pozice:** Pozice uvnitř cyklů grafu, jejichž status nelze jednoznačně určit retrográdní analýzou, zůstávají označeny jako remíza. To nastává, když pozice tvoří silně souvislou komponentu, kde se hráči mohou pohybovat nekonečně dlouho bez jednoznačného vítěze.

Časová složitost je $O(V + E)$, kde $V$ je počet uzlů a $E$ počet hran.

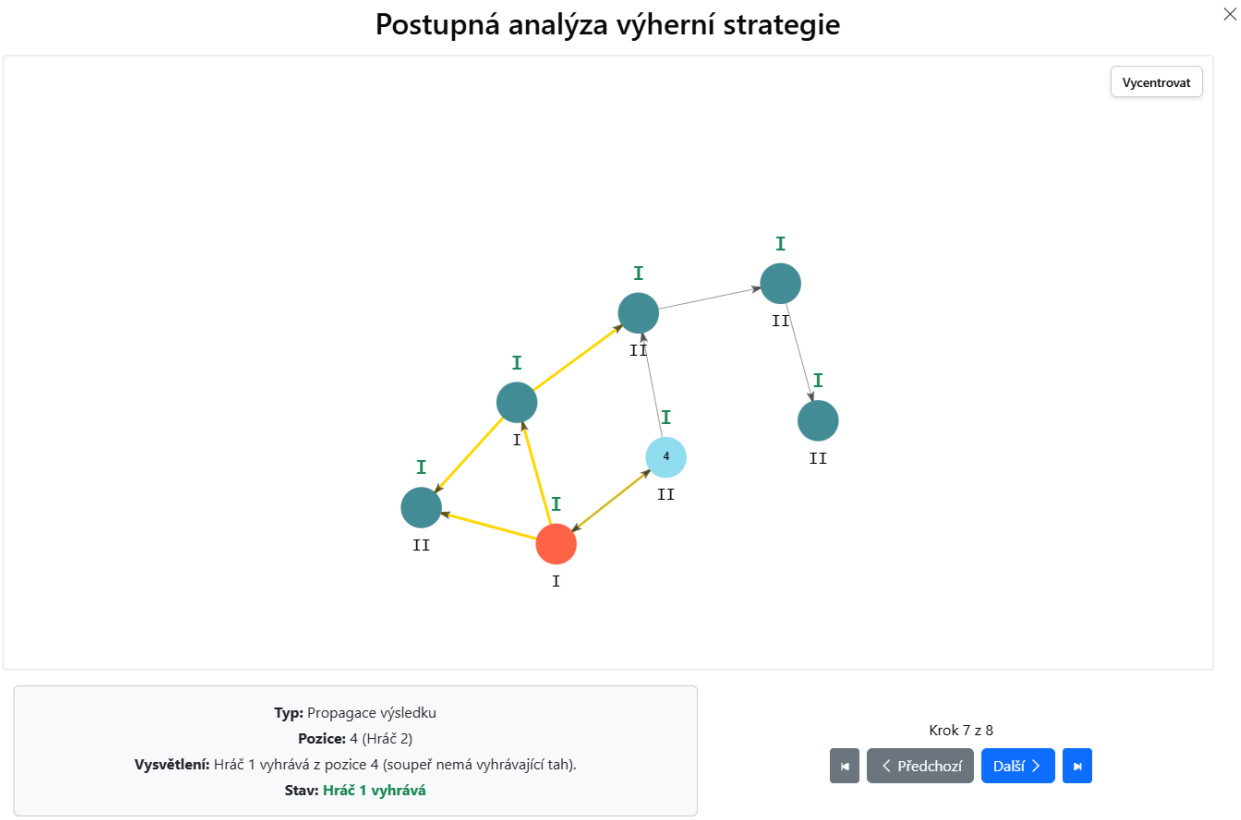
#### 4.2.6.2 Optimální tahy

Funkce `getOptimalMoves()` identifikuje hrany, které jsou součástí výherní strategie [8]. Hrana z pozice $u$ do pozice $v$ je optimální, pokud obě pozice jsou výherní pro Hráče 1. Tyto hrany jsou zvýrazněny ve vizualizaci.

### 4.2.7 Krokové vyhodnocení

Pro detailnější průchod grafem implementuje komponenta `StepByStepGame` krokovatelnou analýzu (viz obrázek 4.7). Algoritmus průběžně zaznamenává změny stavu grafu a uživatel jimi může procházet pomocí navigačních tlačítek. Pro každý krok analýzy je zobrazeno:

- **Aktuálně vyhodnocovaný uzel:** Zvýrazněn přímo v grafu.




Obrázek 4.7: Kroková analýza kombinatorické hry

- **Vysvětlení kroku:** Popis toho, co se v daném kroku děje (nalezení koncové pozice nebo propagace výsledku).
- **Stav z pohledu Hráče 1:** Všechny zprávy konzistentně uvádějí, zda Hráč 1 z dané pozice vyhrává nebo prohrává.
- **Zelené číslo nad uzly:** U vyhodnocených pozic je zobrazen vítězný hráč (1 nebo 2) pro danou pozici.

Po dokončení retrográdní analýzy je přidán závěrečný krok, který zobrazuje celkový výsledek hry – zda Hráč 1 má či nemá výherní strategii z počáteční pozice. Tento finální krok je zvýrazněn žlutým pozadím. Tato funkce pomáhá pochopit, jak algoritmus postupně řeší hru a jak se vypořádává s cykly.

## 4.3 Bezkontextové gramatiky

### 4.3.1 Formát vstupu

Aplikace nabízí tři způsoby zadávání bezkontextové gramatiky:

- **Manuální zadání:** Přímé vkládání pravidel gramatiky v textovém formátu (viz sekce 4.3.2).
- **Generování náhodných gramatik:** Automatické vytvoření gramatiky na základě zadaných parametrů (viz sekce 4.3.3).
- **Předpřipravené sady:** Načtení ukázkových gramatik z knihovny příkladů (viz sekce 4.3.4).

Všechny formy vstupu využívají stejnou komponentu pro zobrazení derivačního stromu gramatiky implementovanou v souboru `DerivationTreeVisual.jsx`. Struktura gramatiky je popsána v sekci 3.3.3 a třídní diagram je na obrázku 3.3.

### 4.3.2 Manuální zadání gramatiky

Komponenta `ManualInput` poskytuje rozhraní ve formě textového okna pro přímé zadávání pravidel gramatiky ve formátu `LS -> PS`. Algoritmus v modulu `GrammarParser.js` automaticky rozpoznává neterminály (symboly začínající velkým písmenem, včetně českých znaků) a terminály (ostatní symboly). To umožňuje používat víceznakové neterminály jako `A1`, `Abc` nebo `A_10`.

Parser akceptuje jak zápis šipky `->` z klávesnice, tak i jako symbol `→`. Obě varianty jsou interně sjednoceny na stejnou reprezentaci pravidla, což zjednodušuje ruční zadávání gramatiky.

### 4.3.3 Generování náhodných gramatik

Modul `GrammarGenerator.js` obsahuje funkci `generateGrammar()` pro vytváření náhodných bezkontextových gramatik na základě vstupních parametrů.

#### 4.3.3.1 Základní parametry

Základní parametry určují počet neterminálů (1–10), počet terminálů (1–10) a maximální délku pravé strany (1–5).

#### 4.3.3.2 Pokročilé parametry

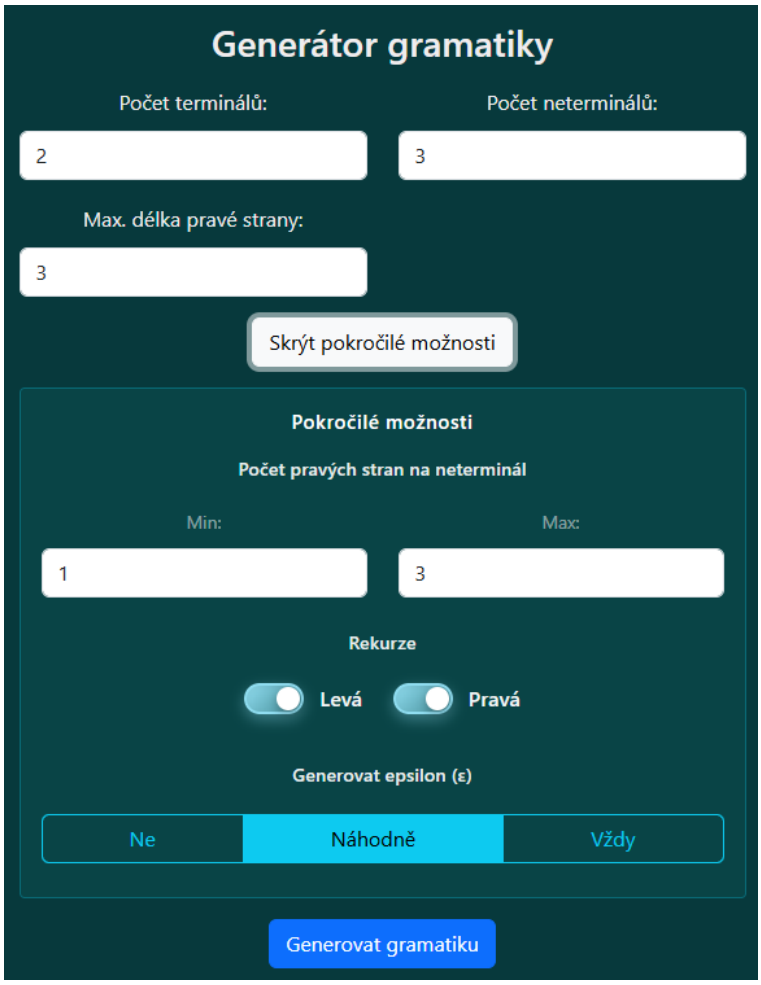
Pokročilé parametry umožňují jemnější kontrolu nad vlastnostmi generované gramatiky (viz obrázek 4.8):

- **Počet pravidel na neterminál:** Parametry `minProductionsPerNonTerminal` a `maxProductionsPerNonTerminal` určují rozsah, ve kterém se bude pohybovat počet pravidel pro každý neterminál. Výchozí hodnoty jsou minimálně 1 a maximálně 3.
- **Levá a pravá rekurze:** Zapnutí parametru `allowLeftRecursion` umožňuje vytvářet pravidla typu $A \to A\alpha$, zatímco `allowRightRecursion` povoluje pravidla typu $A \to \alpha A$. Oba typy rekurze mohou být přítomny současně, a mohou také být aplikovány na jediné pravidlo, což vede ke vzniku centrální rekurze typu $A \to A\alpha A$. Při každém generování pravidla je pravděpodobnost použití levé i pravé rekurze 30%.
- **Režim epsilon pravidel:** Parametr `epsilonMode` nabízí tři možnosti:
  - `'Ne'`: Žádné epsilon pravidlo nebude vygenerováno.
  - `'Náhodně'`: Epsilon pravidla vznikají náhodně s malou pravděpodobností (8%).
  - `'Vždy'`: Algoritmus zaručí, že alespoň jedno epsilon pravidlo bude v gramatice přítomno.

#### 4.3.3.3 Algoritmus generování

Algoritmus implementovaný ve funkci `generateGrammar()` prochází několika fázemi:

1. **Generování symbolů:**
   - Neterminály začínají velkým písmenem. První symbol je vždy `S` (startovní symbol), následují písmena `A`–`Z`, a při potřebě více symbolů pokračuje kombinacemi `AA`, `AB` atd.
   - Terminály začínají malým písmenem, konkrétně se používají písmena `a`–`z`.
2. **Vytvoření pravidel:** Pro každý neterminál se vygeneruje náhodný počet pravidel v rozsahu určeném parametry `minProductionsPerNonTerminal` a `maxProductionsPerNonTerminal`.
3. **Tvorba pravé strany pravidla:** Každé pravidlo vzniká následovně:
   - Podle režimu epsilon může být s určitou pravděpodobností vygenerováno prázdné pravidlo ($\varepsilon$).




Obrázek 4.8: Rozhraní pro nastavení parametrů generování bezkontextových gramatik

- Na základě parametrů levé a pravé rekurze se rozhodne, zda bude pravidlo obsahovat rekurzivní volání aktuálního neterminálu na začátku nebo konci.
- Pro zbývající část pravidla se náhodně vybírají symboly, přičemž s pravděpodobností 50% se vkládá terminál a s pravděpodobností 50% neterminál (kromě aktuálního, aby nedošlo k nechtěné přímé rekurzi mimo povolenou levou a pravou).
- Maximální délka této nerekurzivní části odpovídá hodnotě parametru `maxRuleLength` snížené o počet již přidaných rekurzivních symbolů.

4. **Zajištění dosažitelnosti:** Po vygenerování všech pravidel algoritmus provede průchod do šířky (BFS) ze startovního symbolu `S` a identifikuje všechny dosažitelné neterminály. Pro každý nedosažitelný neterminál následně náhodně vybere pravidlo některého dosažitelného neterminálu a upraví jej tak, aby odkazovalo na dosud nedosažitelný symbol. Tím je zaručeno, že všechny neterminály jsou využitelné v derivaci.

5. **Zajištění epsilon pravidla:** V režimu `epsilonMode='always'` (uživatelská volba „Vždy“) algoritmus ověří, že alespoň jeden neterminál obsahuje epsilon pravidlo. Pokud ne, náhodně zvolí neterminál a přidá mu prázdné pravidlo.

Výsledná gramatika je tak validní bezkontextová gramatika s dosažitelnými neterminály a kontrolovanými vlastnostmi rekurze.

### 4.3.4 Předpřipravené sady

Ve složce `Sady/Grammar` jsou uloženy předpřipravené příklady gramatik ve formátu JSON (viz obrázek 4.9), jehož struktura je popsána v sekci 3.3.3.

### 4.3.5 Algoritmus vyhodnocení

Modul `GrammarEvaluator.js` obsahuje implementaci algoritmu pro vyhodnocení neprázdnosti gramatiky založenou na teorii popsané v sekci 2.4.

#### 4.3.5.1 Identifikace produktivních neterminálů

Algoritmus pracuje iterativně a postupně označuje produktivní neterminály. Implementace používá optimalizovanou verzi s frontou (work-list algorithm), což odpovídá principu BFS. Postup algoritmu:

1. **Inicializace:** Algoritmus vytvoří prázdnou množinu produktivních neterminálů a frontu pro zpracování. Všechna pravidla jsou zpočátku neoznačená.

2. **Nalezení základních produktivních pravidel:** Projdou se všechna pravidla gramatiky a identifikují se ta, jejichž pravá strana obsahuje pouze terminály nebo je prázdná ($\varepsilon$). Neterminály na levé straně těchto pravidel jsou okamžitě označeny jako produktivní a vloženy do

```
1  {
2    "name": "Základní sčítání",
3    "nonTerminals": ["S", "E"],
4    "terminals": ["+", "id"],
5    "productions": {
6      "S": [
7        ["E"]
8      ],
9      "E": [
10       ["E", "+", "id"],
11       ["id"]
12     ]
13   }
14 }
```

Obrázek 4.9: Ukázka formátu JSON pro uložení bezkontextové gramatiky

fronty. Pokud nebylo nalezeno žádné takové pravidlo, fronta zůstane prázdná a další krok iterativní propagace se neprovede vůbec (přeskočí se krok 3.).

3. **Propagace produktivity:**
   - Vyjme se neterminál z fronty.
   - Projdou se všechna dosud neoznačená pravidla gramatiky.
   - Pro každé pravidlo se zkontroluje, zda jsou všechny symboly na pravé straně buď terminály, nebo již označené produktivní neterminály.
   - Pokud ano, neterminál na levé straně tohoto pravidla je označen jako produktivní, pravidlo se označí jako zpracované a neterminál je přidán do fronty (pokud tam ještě není).
   - Pokud ne (tedy pravá strana obsahuje alespoň jeden dosud neproduktivní neterminál), pravidlo zůstává neoznačené a bude znovu zkontrolováno v dalších iteracích.
4. **Ukončení:** Algoritmus končí, když je fronta prázdná. V tu chvíli jsou všechny produktivní neterminály nalezeny.

Časová složitost je $O(|P| \cdot |N|)$, kde $|P|$ je počet pravidel gramatiky a $|N|$ je počet neterminálů. V nejhorším případě je každé pravidlo kontrolováno jednou pro každý neterminál, který se stane produktivním.

#### 4.3.5.2 Rekonstrukce derivace

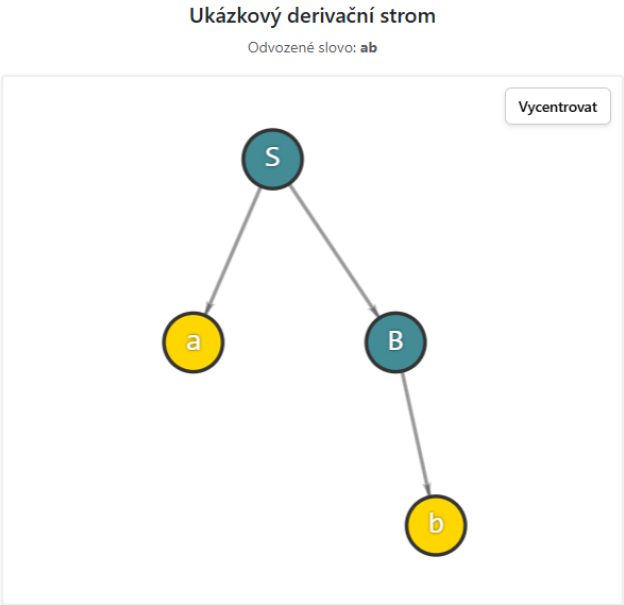
Pokud je počáteční symbol $S$ produktivní, aplikace sestaví ukázkový derivašní strom. Nejprve se z množiny produktivních pravidel vypočítá pro každý neterminál odhad minimální délky odvoditel-

ného terminálního řetězce. Tento krok opakovaně přepočítává hodnoty pravidel, dokud se už nemění (fixed-point výpočet), a vychází ze stejné myšlenky jako algoritmy pro hledání nejkratší cesty [4].

Na základě vypočtených hodnot se pro každý produktivní neterminál vybere pravidlo s nejmenší cenou (a při shodě pravidlo s menším počtem symbolů na pravé straně). Tím vznikne malá konečná svědecká derivace, která upřednostňuje krátké a přehledné příklady. Samotná konstrukce stromu je pak provedena iterativně pomocí zásobníku namísto hluboké rekurze, aby byla robustní i pro gramatiky s výraznou rekurzí. Použitý postup navazuje na standardní fixpoint/work-list přístup při analýze bezkontextových gramatik [9].

### 4.3.6 Vizualizace derivačního stromu

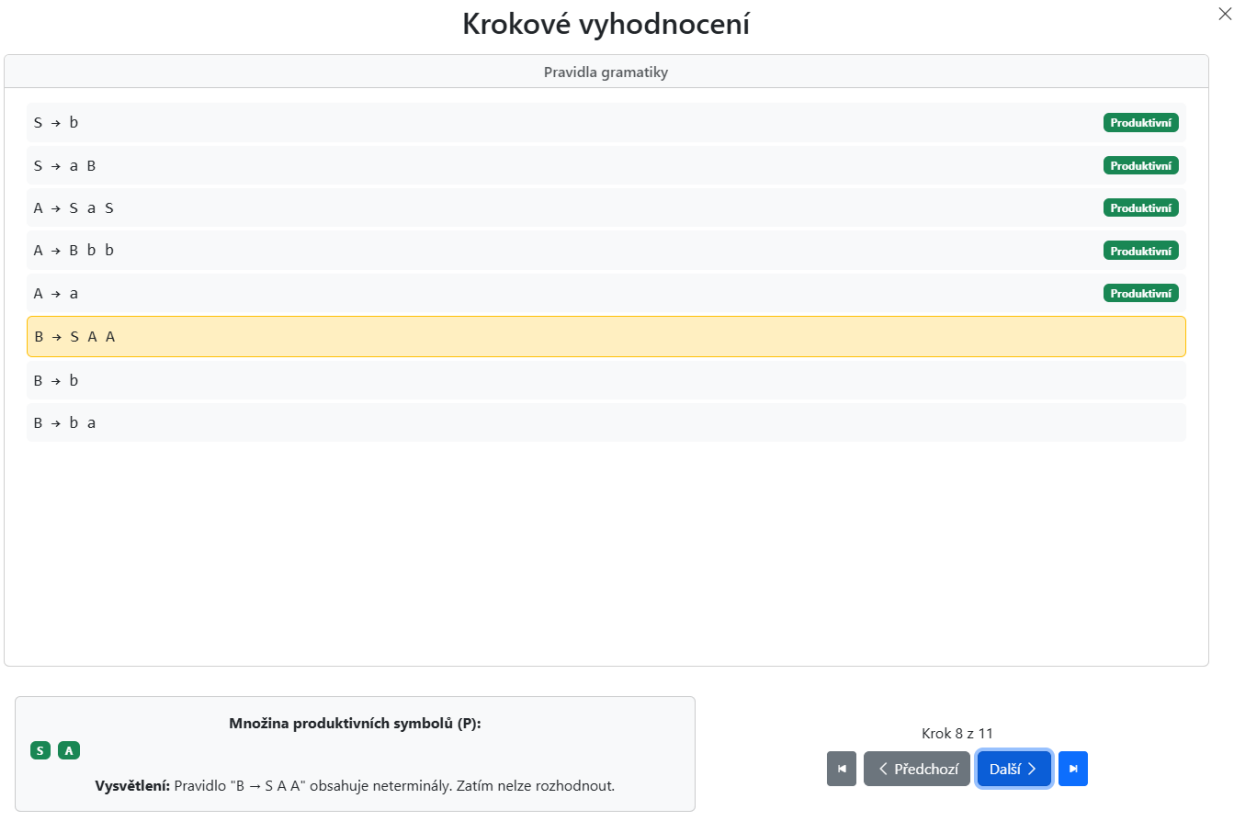
Pokud gramatika generuje neprázdný jazyk, aplikace vizuálně ukáže derivační strom pomocí komponenty `DerivationTreeVisual` (viz obrázek 4.10).




Obrázek 4.10: Vizualizace derivačního stromu pro neprázdnou gramatiku

### 4.3.7 Krokové vyhodnocení

Komponenta `GrammarStepEvaluator.js` vypočítá jednotlivé kroky pro komponentu `StepByStepGrammar`, která je následně interaktivně zobrazí. Zobrazuje seznam pravidel s barevným zvýrazněním aktuálního stavu a množinu dosud nalezených produktivních symbolů (viz obrázek 4.11).




Obrázek 4.11: Kroková analýza bezkontextové gramatiky

## 4.4 Implementace převodů mezi problémy

Pro ilustraci vzájemné převoditelnosti P-úplných problémů popsané v sekci 2.5 aplikace implementuje převody z MCVP na ostatní dva problémy.

### 4.4.1 Převod na kombinatorickou hru

Třída `MCVPToGameStepGenerator` implementuje převod pomocí rekurzivního průchodu MCVP grafem. Pro každý uzel vytváří odpovídající pozici ve hře dle pravidel definovaných v sekci 2.5.1.

#### 4.4.1.1 Implementační detaily

Algoritmus využívá několik klíčových mechanismů:

1. **Generování identifikátorů:** Každý uzel MCVP obvodu obdrží unikátní identifikátor ve formě `V0`, `V1`, `V2` atd. pomocí mapy `labelMap`, která zajišťuje konzistentní přiřazení – tentýž uzel vždy obdrží stejný identifikátor. To je důležité pro správné propojení pozic v případě sdílených podgrafů.

2. **Sledování navštívených uzlů:** Množina `visited` zabraňuje duplicitnímu zpracování uzlů. Pokud algoritmus narazí na již navštívený uzel, rekurzivní průchod se okamžitě ukončí, protože odpovídající herní pozice již byla vytvořena. Tím je zajištěno, že každá pozice v herním grafu existuje právě jednou.

3. **Ponechání zpětných odkazů:** Pro každou pozici jsou kromě seznamu následovníků (`children`) udržovány i zpětné odkazy na předchůdce (`parents`). Tyto zpětné odkazy umožňují efektivní retrográdní analýzu, která propaguje výsledky směrem od terminálních pozic k počáteční pozici.

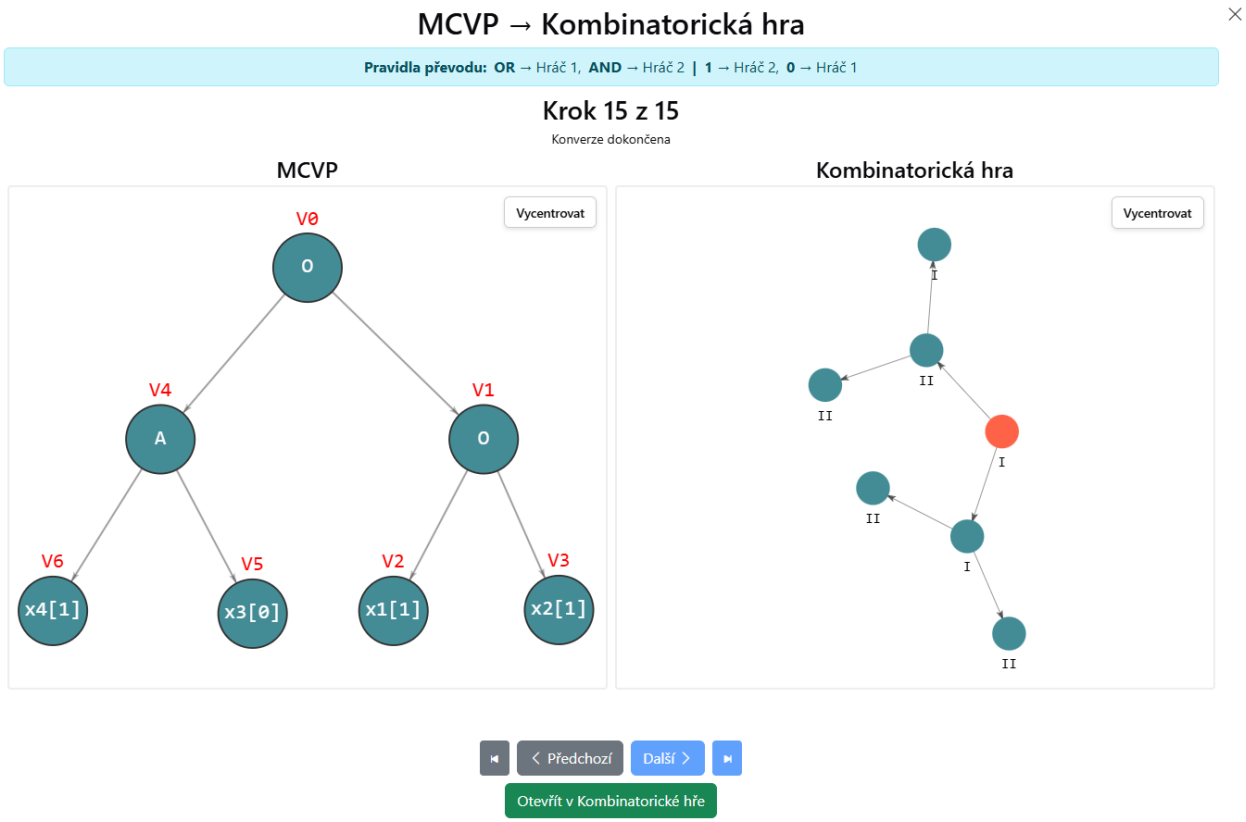
4. **Krokování převodu:** Pro každou důležitou operaci (vytvoření pozice, propojení pozic) je zaznamenán krok obsahující popis akce, zvýrazněný uzel a aktuální stav grafu. Tyto kroky umožňují komponentě postupně vizualizovat celý proces převodu.

5. **Určení počáteční pozice:** Kořen MCVP obvodu (výstupní uzel DAGu) se automaticky stává počáteční pozicí hry, která je ve vizualizaci označena odlišnou barvou.

Vizualizace probíhá pomocí komponenty `MCVPtoCombinatorialGameConverter`, která zobrazuje převod krokovatelně (viz obrázek 4.12).

### 4.4.2 Převod na bezkontextovou gramatiku

Převod implementuje třída `MCVPToGrammarConverter` dle principů ze sekce 2.5.2. Algoritmus probíhá ve dvou fázích: nejprve přiřadí uzlům unikátní symboly a poté vytvoří odpovídající pravidla.




Obrázek 4.12: Ukázka převodu MCVP obvodu na kombinatorickou hru

#### 4.4.2.1 Generování symbolů gramatiky

Pro vytváření symbolů jsou použity dvě pomocné třídy:

- **NonTerminalGenerator:** Generuje neterminální symboly ve formě velkých písmen. Kořen obvodu vždy obdrží symbol `S` (startovní symbol gramatiky). Pro další uzly jsou použity symboly `A`–`Z`, a pokud je potřeba více než 26 symbolů, pokračuje se kombinacemi `AA`, `AB`, `AC` až `ZZ`.
- **TerminalGenerator:** Pro každou proměnnou s hodnotou 1 generuje jedinečný terminální symbol. Postupně jsou použity znaky `a`–`z`, následně speciální znaky (`+`, `-`, `=`, `*` atd.), a pokud je symbolů nedostatek, pokračuje se číslovanými terminály `t1`, `t2` atd.

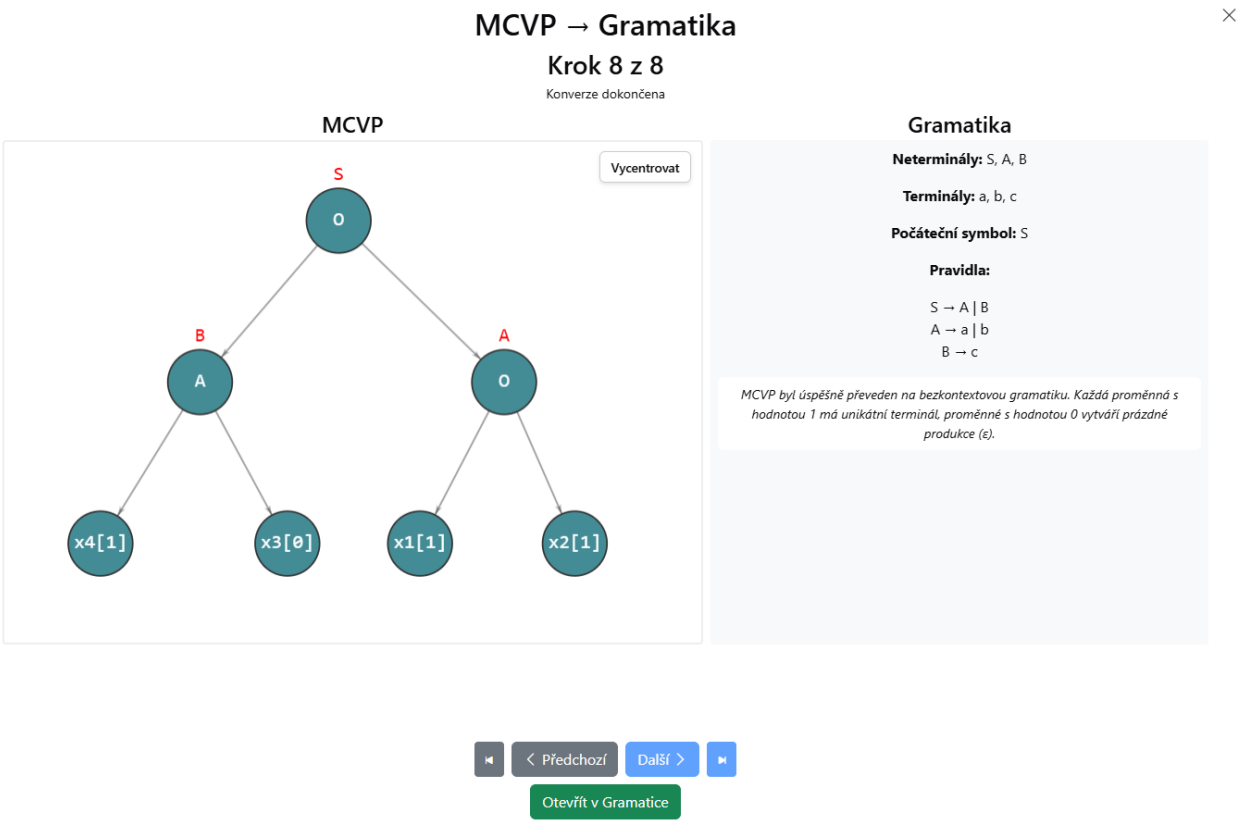
Vstupní proměnné (`variable`) (kromě případu, kdy je proměnná kořenem) **nedostávají vlastní neterminální symbol**. Místo toho jsou při tvorbě pravidel nahrazeny přímo terminály nebo prázdným řetězcem.

#### 4.4.2.2 Tvorba produkčních pravidel

Druhá fáze algoritmu prochází DAGem a vytváří pravidla:

1. **Hradlo AND:** Vytvoří pravidlo ve formě `X → AB...`, kde `X` je neterminál odpovídající hradlu a `A`, `B` jsou symboly potomků. Pokud je potomek proměnná s hodnotou 1, je do pravidla vložen odpovídající terminál. Pokud je potomek proměnná s hodnotou 0, tento potomek je při tvorbě pravidla zcela vynechán. Speciálně, pokud jsou všichni potomci proměnné s hodnotou 0, vzniká pravidlo `X →` $\varepsilon$.
2. **Hradlo OR:** Vytvoří několik alternativních pravidel, vždy jedno pro každého potomka: `X → A`, `X → B` atd. Pokud je potomek proměnná s hodnotou 0, vzniká pravidlo `X →` $\varepsilon$.
3. **Proměnná jako kořen:** Tento speciální případ nastává, když celý MCVP obvod obsahuje pouze jednu proměnnou. Pokud má proměnná hodnotu 1, je vytvořeno pravidlo `S → t` pro nějaký terminál `t`. Pokud má hodnotu 0, vzniká pravidlo `S →` $\varepsilon$.

Výsledná gramatika přesně odpovídá hodnotě MCVP obvodu: gramatika generuje neprázdný jazyk právě tehdy, když obvod vyhodnotí hodnotu 1. Vizualizace probíhá pomocí komponenty `MCVPtoGrammarConverter`, která zobrazuje převod krokovatelně (viz obrázek 4.13).




Obrázek 4.13: Ukázka převodu MCVP obvodu na bezkontextovou gramatiku

# Kapitola 5

# Závěr

Cílem této práce bylo vytvořit interaktivní webovou aplikaci pro vizualizaci a demonstraci převoditelnosti a řešení P-úplných problémů na příkladu tří vybraných P-úplných problémů: Monotone Circuit Value Problem, kombinatorických her na grafu a problému neprázdnosti jazyka bezkontextové gramatiky.

## 5.1 Dosažené výsledky

Výsledná aplikace splňuje všechny stanovené cíle a poskytuje jednotné prostředí pro práci se třemi P-úplnými problémy. Pro každý problém je implementováno kompletní řešení zahrnující:

- **Flexibilní vstupní způsoby:** Uživatelé mohou zadávat instance problémů třemi způsoby – manuálním zadáním, generováním náhodných instancí nebo pomocí předpřipravených příkladů.
- **Vizualizaci řešení:** Každý problém je doprovázen grafickou reprezentací, která využívá knihovnu *react-force-graph-2d* pro interaktivní zobrazení grafových struktur. Uživatel může manipulovat se zobrazenými grafy, přibližovat je a přesouvat uzly pro lepší orientaci.
- **Krokové vyhodnocení:** Implementace krokovatelného průchodu algoritmů umožňuje sledovat každý jednotlivý krok výpočtu s textovým vysvětlením.
- **Export a import dat:** Možnost ukládání a načítání instancí problémů ve formátu JSON podporuje sdílení příkladů a vytváření připravených příkladů.

Aplikace taktéž demonstruje dva konkrétní převody z MCVP na kombinatorickou hru a na bezkontextovou gramatiku. Oba převody jsou implementovány krokovatelně, takže uživatel může sledovat, jak se jednotlivé uzly obvodu transformují na odpovídající struktury v cílovém problému. Tato vizualizace ilustruje techniku polynomiálních redukcí a vzájemnou převoditelnost P-úplných problémů.

## 5.2 Vzdělávací přínos

Aplikace představuje vzdělávací nástroj pro výuku teorie složitosti s interaktivním přístupem. Krokové vyhodnocení s textovými vysvětleními umožňuje pochopit fungování algoritmů na konkrétních příkladech, zatímco vizualizace převodů demonstruje vzájemnou převoditelnost P-úplných problémů. Generování náhodných instancí pak podporuje experimentální pozorování chování algoritmů na různých instancích problémů.

## 5.3 Možnosti dalšího rozvoje

Přestože aplikace poskytuje funkční implementaci všech plánovaných funkcí, existuje prostor pro další rozšíření:

- **Další P-úplné problémy:** Aplikace by mohla být rozšířena o další P-úplné problémy, jako je například vyhodnocování booleovských formulí v konjunktivní normální formě nebo problém dosažitelnosti v grafech s omezenou šířkou.
- **Více převodů:** Implementace dalších směrů převodů – například z kombinatorických her na gramatiky nebo opačným směrem z gramatik na MCVP – by poskytla kompletnější obraz vzájemné převoditelnosti těchto problémů.

Výsledná aplikace může sloužit jako doplněk k tradičním výukovým materiálům v kurzech teorie složitosti a teoretické informatiky, kde pomáhá studentům lépe pochopit abstraktní koncepty prostřednictvím konkrétních interaktivních příkladů.

# Literatura


1. ARORA, Sanjeev; BARAK, Boaz. *Computational Complexity: A Modern Approach*. Cambridge University Press, 2009. ISBN 978-0521424264.
2. ASTURIANO, Vasco. *react-force-graph: React component for 2D, 3D, VR and AR force directed graphs [online]* [GitHub]. 2024. Dostupné také z: `https://github.com/vasturiano/react-force-graph`. [cit. 2026-02-03].
3. BOOTSTRAP TEAM. *Bootstrap: Build fast, responsive sites [online]* [OpenJS Foundation]. 2024. Dostupné také z: `https://getbootstrap.com/`. Verze 5.3.8. [cit. 2026-02-03].
4. CORMEN, Thomas H.; LEISERSON, Charles E.; RIVEST, Ronald L.; STEIN, Clifford. *Introduction to Algorithms*. 3rd. MIT Press, 2009. ISBN 978-0262033848.
5. META PLATFORMS, INC. *React: The library for web and native user interfaces [online]* [Meta Open Source]. 2024. Dostupné také z: `https://react.dev/`. Verze 19.2. [cit. 2026-02-03].
6. MIYANO, Satoru; SHIRAISHI, Shunsuke; SHOUDAI, Takayoshi. *A List of P-Complete Problems [online]*. 1990-12. Tech. zpr., RIFIS-TR-CS-17. Fukuoka: Kyushu University, Research Institute of Fundamental Information Science. Dostupné také z: `https://catalog.lib.kyushu-u.ac.jp/opac_download_md/3123/rifis-tr-17.pdf`. [cit. 2026-02-03].
7. PAPADIMITRIOU, Christos H. *Computational Complexity*. Addison Wesley, 1994. ISBN 978-0201530827.
8. SAWA, Zbyněk. *Teoretická informatika – Podklady pro přednášky [online]* [Ostrava: VŠB-TUO, Katedra informatiky]. 2025. Dostupné také z: `https://www.cs.vsb.cz/sawa/ti/slides/ti-slides-08.pdf`. [cit. 2026-02-03].
9. SAWA, Zbyněk. *Úvod do teoretické informatiky – Bezkontextové gramatiky [online]* [Ostrava: VŠB-TUO, Katedra informatiky]. 2014. Dostupné také z: `https://www.cs.vsb.cz/kot/download/uti2014/uti-10-cz.pdf`. [cit. 2026-02-03].
10. SAWA, Zbyněk. *Úvod do teoretické informatiky – Turingovy stroje [online]* [Ostrava: VŠB-TUO, Katedra informatiky]. 2014. Dostupné také z: `https://www.cs.vsb.cz/kot/download/uti2014/uti-08-cz.pdf`. [cit. 2026-02-03].

11. SAWA, Zbyněk. *Úvod do teoretické informatiky – Výpočetní složitost algoritmů [online]* [Ostrava: VŠB-TUO, Katedra informatiky]. 2014. Dostupné také z: `https://www.cs.vsb.cz/kot/download/uti2014/uti-12-cz.pdf`. [cit. 2026-02-03].

12. VITE TEAM. *Vite: The Build Tool for the Web [online]* [VoidZero Inc.]. 2024. Dostupné také z: `https://vite.dev/`. Verze 6. [cit. 2026-02-03].